

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Disciplina: Teste de Software

Professor: Cleiton Silva Tavares

Trabalho: Bad Smells

Aluno: Raphael Sena Augusto de Brito

Matrícula: 699651

Data: 16 de maio de 2026

1. Análise de Smells

Bad Smell 1: Complexidade Cognitiva Excessiva (Cognitive Complexity)

Localização: ReportGenerator.js , método: generateReport()

O método generateReport() possuía complexidade cognitiva de 31 pontos, configurada para máximo de 5 pontos. Isso significa que o método continha múltiplas camadas de aninhamento e lógica condicional entrelaçada:

- Loop for para iterar itens
- Condicional if/else if para diferenciar ADMIN vs USER
- Condicional aninhada para verificar valor limite (>1000)
- Condicionais aninhadas para formatar por tipo (CSV vs HTML)

Por que é problemático:

- Manutenção: Fazer uma mudança em uma lógica requer entender toda a complexidade
- Testes: Fica muito difícil alcançar cobertura completa de todos os caminhos
- Bugs: Alterações em uma parte podem quebrar comportamentos em outras
- Legibilidade: Novos desenvolvedores precisam de muito tempo para compreender a lógica

Bad Smell #2: If Statement Colapsável (No Collapsible If)

Localização: ReportGenerator.js

```
if (item.value <= 500) {  
  if (reportType === 'CSV') {  
    // ... código  
  }  
}
```

Dois if statements consecutivos que poderiam ser combinados em uma única condição com &&.

Por que é problemático:

- Legibilidade: Aumenta desnecessariamente a profundidade de indentação

- Duplication: A mesma lógica se repete em múltiplas partes
- Manutenção: Modificar a lógica de validação requer mudanças em múltiplos lugares

2. Relatório da Ferramenta

```
PS E:\dev\repositories\pucminas\2026\1\teste\bad-smells-js-refactoring> npx eslint src/

E:\dev\repositories\pucminas\2026\1\teste\bad-smells-js-refactoring\src\ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 31 to the 5 allowed  sonarjs/cognitive-complexity
  45:14  error  Merge this if statement with the nested one  sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

O eslint-plugin-sonarjs foi essencial, porque detectou problemas não-óbvios: A complexidade cognitiva é uma métrica sofisticada que mede a dificuldade mental de ler o código, não apenas o número de linhas. Uma análise manual poderia ter classificado o método como "apenas longo" em vez de identificar a verdadeira causa raiz (aninhamento excessivo). Além disso, estabeleceu um limite claro (5 pontos) e mostrou exatamente quanto o código estava fora desse limite (31 pontos = 6x pior). Sem a ferramenta, seria difícil argumentar quando o código está "ruim demais".

Ele pôde identificar Patterns ao padrão de "if colapsável" automaticamente, que um revisor humano poderia ter perdido em uma revisão superficial. E garantir que todo o time siga os mesmos critérios de qualidade, não dependendo do julgamento individual.

3. Processo de Refatoração

O smell mais crítico que corrigi foi a complexidade cognitiva excessiva no método generateReport(). No arquivo original, ReportGenerator.js, esse método concentrava toda a lógica de geração do relatório: montagem do cabeçalho, filtro por tipo de usuário, formatação dos itens e geração do rodapé. Isso deixava o código difícil de ler, testar e modificar.

```
generateReport(reportType, user, items) {
  let report = "";
  let total = 0;

  if (reportType === 'CSV') {
    report += 'ID,NOME,VALOR,USUARIO\n';
  } else if (reportType === 'HTML') {
    report += '<html><body>\n';
    report += '<h1>Relatório</h1>\n';
    report += '<h2>Usuário: ${user.name}</h2>\n';
    report += '<table>\n';
    report += '<tr><th>ID</th><th>Nome</th><th>Valor</th></tr>\n';
  }

  for (const item of items) {
    if (user.role === 'ADMIN') {
      if (item.value > 1000) {
```

```

        item.priority = true;
    }

    if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
    } else if (reportType === 'HTML') {
        const style = item.priority ? 'style="font-weight:bold;"' : '';
        report += style
            ? `<tr
${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`
            : `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
        total += item.value;
    }
} else if (user.role === 'USER') {
    if (item.value <= 500) {
        if (reportType === 'CSV') {
            report += `${item.id},${item.name},${item.value},${user.name}\n`;
            total += item.value;
        } else if (reportType === 'HTML') {
            report +=
`<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
            total += item.value;
        }
    }
}
}

if (reportType === 'CSV') {
    report += '\nTotal,.\n';
    report += `${total},.\n`;
} else if (reportType === 'HTML') {
    report += '</table>\n';
    report += `<h3>Total: ${total}</h3>\n`;
    report += '</body></html>\n';
}

return report.trim();
}

```

Depois:

```

generateReport(reportType, user, items) {
    let report = this.getHeader(reportType, user);
    const filteredItems = this.filterItemsByRole(user, items);
    const { content, total } = this.formatItems(reportType, user, filteredItems);
}

```

```

report += content;
report += this.getFooter(reportType, total);

return report.trim();
}

```

4. Conclusão

Durante a refatoração, os testes foram absolutamente essenciais. Ao invés de modificar o código original diretamente (o que teria sido arriscado), eu:

1. Criei um novo arquivo ReportGenerator.refactored.js
2. Criei um arquivo de testes correspondente
3. Implementei a refatoração incrementalmente
4. Rodei os testes após cada mudança
5. Apenas quando todos os 5 testes passaram, considerei a refatoração completa
6. Sem os testes, não teria confiança de que a nova implementação produz exatamente a mesma saída que a antiga.
7. Os testes atuam como um "contrato" que garante a equivalência comportamental.

Impacto da Redução de Bad Smells na Qualidade do Software

Aspecto	Antes	Depois
Manutenibilidade	Difícil (30+ linhas + 4 níveis de aninhamento)	Fácil (7 métodos pequenos + 2 níveis máxim)
Testabilidade	Difícil (muitos caminhos de código)	Fácil (cada método testável isoladamente)
Legibilidade	Ruim (complexidade cognitiva 31)	Excelente (complexidade cognitiva 4)
Risco de Bugs	Alto (mudanças afetam múltiplos caminhos)	Baixo (métodos isolados)